MCMP117 · WEEK 13

Fundamentals of Operating Systems

Week 13 · Processes & Job Control

Every command you type becomes a process. Today you learn to watch it, pause it, move it and stop it.

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 12 in Four Ideas

Three streams

Every process starts with fd 0 (stdin), 1 (stdout) and 2 (stderr).

Redirection

> >> < 2> 2>&1 &> /dev/null — done by the shell between fork() and exec().

Pipes

cmd1 | cmd2 runs two processes at the same time, joined by a kernel buffer.

Filters

grep, sort, uniq -c, cut, wc, tr, head, tail: small tools combined into pipelines.

Last week the shell quietly created processes for every pipeline. Today we look at those processes directly — and take control of them.

Today's Agenda

1. **Looking at processes** ps, pstree, top, /proc/<pid>/status — and the STAT letters R S D T Z.
2. **Job control** Foreground and background, &, Ctrl+Z, Ctrl+C, jobs, fg, bg.
3. **Signals** kill, kill -l, SIGTERM vs SIGKILL vs SIGINT, SIGSTOP/SIGCONT, pkill, killall, nohup.
4. **Priority** nice and renice: the nice values of Week 5, measured.
5. **Zombies** A small C program that makes state Z visible.
6. **Lab** Start, inspect, suspend, resume and kill jobs (graded exercises).

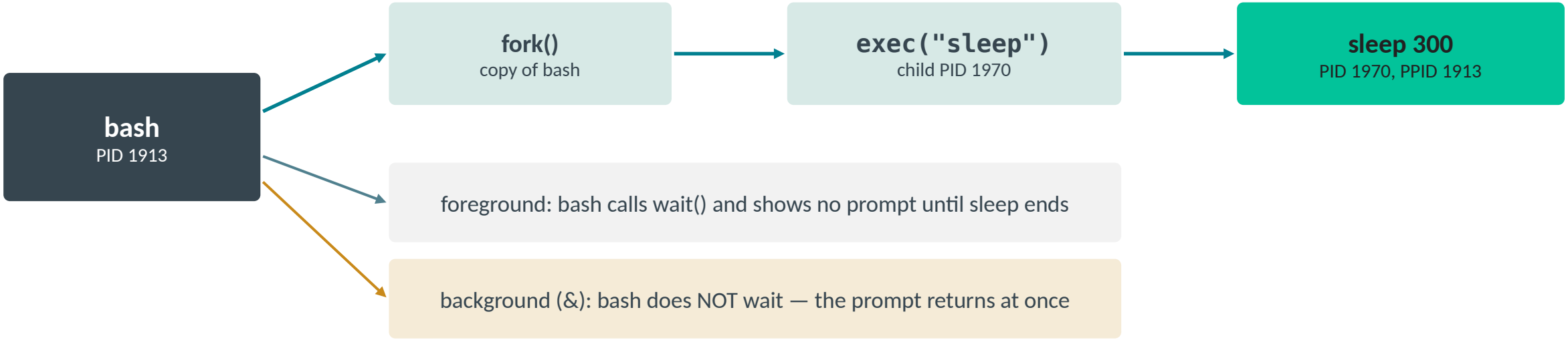
PART 1

Looking at Processes

Week 3 on paper — Week 13 in the terminal.

Every Command Is a Process: fork, exec, wait

A process is a program in execution, with a PID, a parent (PPID) and a PCB in the kernel. The shell creates one for each command.

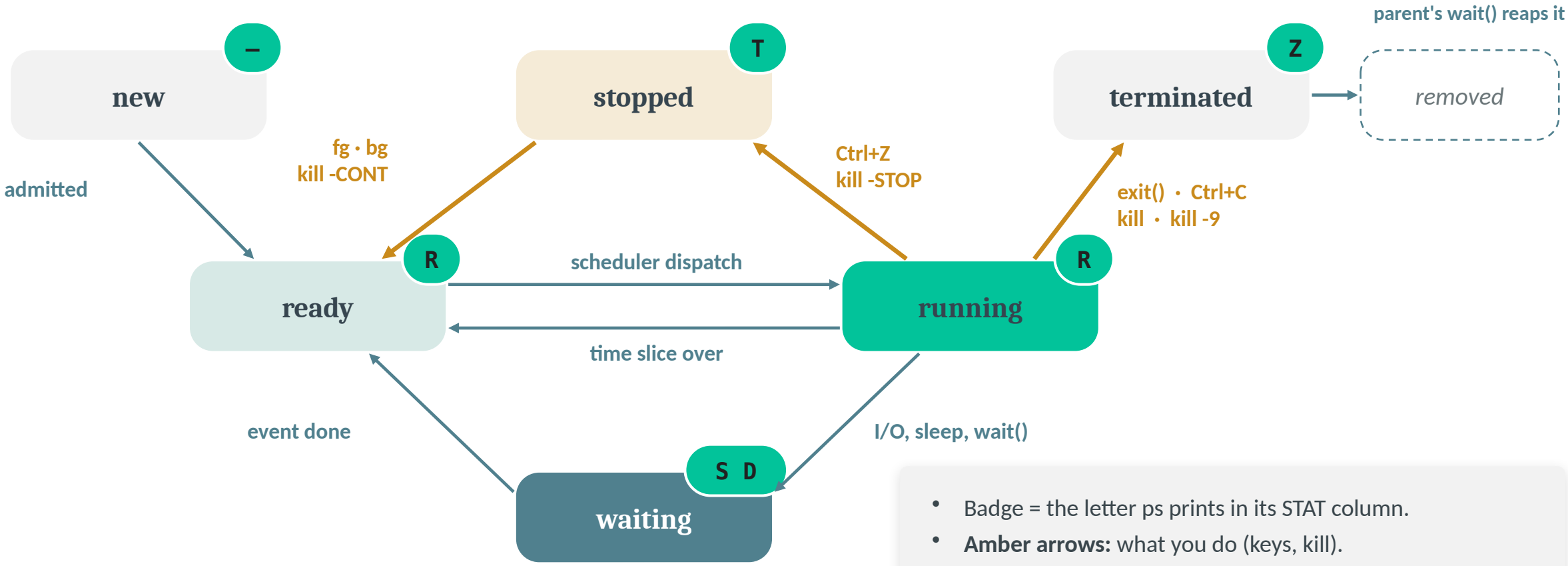



```
student@okan-lab:~/lab$ sleep 300 &
[1] 1970
student@okan-lab:~/lab$ pstree -p $$
bash(1913)─┬─pstree(1975)
           ├─sleep(1970)
           └─yes(1971)
```

Same PID before and after exec(): the child keeps its number but runs a new program. [1] is the job number, 1970 the PID.

LINK TO WEEK 3

Week 3 States, ps STAT Letters, and What You Can Do



- Badge = the letter ps prints in its STAT column.
- **Amber arrows:** what you do (keys, kill).
- **Grey arrows:** the kernel and scheduler (Week 3).

Linux adds stopped (T). R = ready OR running.

PS STAT COLUMN

The STAT Letters and Their Extra Flags

Letter	Meaning	Week 3 state	Seen today in
R	running or runnable (on a CPU or in the ready queue)	running / ready	yes > /dev/null
S	interruptible sleep: waiting for an event	waiting	sleep 300, bash
D	uninterruptible sleep, usually disk I/O	waiting	(rare, short-lived)
T	stopped by Ctrl+Z or SIGSTOP	— (Linux extra)	after Ctrl+Z
Z	zombie: finished, not yet reaped	terminated	./zombie (Part 5)

Extra flags after the letter: s = session leader (your login shell: Ss), + = in the foreground group (R+), N = low priority (niced: RN), < = high priority, l = multithreaded.

ps: A Snapshot of Your Processes

```
student@okan-lab:~/lab$ echo $$
1913
student@okan-lab:~/lab$ ps
  PID TTY          TIME CMD
 1913 pts/0    00:00:00 bash
 1969 pts/0    00:00:00 ps
student@okan-lab:~/lab$ sleep 300 &
[1] 1970
student@okan-lab:~/lab$ yes > /dev/null &
[2] 1971
student@okan-lab:~/lab$ ps
  PID TTY          TIME CMD
 1913 pts/0    00:00:00 bash
 1970 pts/0    00:00:00 sleep
 1971 pts/0    00:00:00 yes
 1972 pts/0    00:00:00 ps
student@okan-lab:~/lab$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   1913  1912  0 16:58 pts/0    00:00:00 -bash
student   1970  1913  0 16:58 pts/0    00:00:00 sleep 300
student   1971  1913 96 16:58 pts/0    00:00:00 yes
student   1973  1913  0 16:58 pts/0    00:00:00 ps -f
```

What to observe

- echo $$ prints the PID of the shell itself: 1913.
- Plain ps: processes of this terminal only (TTY pts/0).
- ps is in its own list — it is a process too.
- ps -f (full): adds UID, PPID, start time, the whole command line.
- PPID of sleep and yes = 1913: the shell is their parent.

Choose Your Columns; Draw the Tree

```
student@okan-lab:~/lab$ ps -o pid,ppid,stat,ni,%cpu,%mem,cmd
  PID  PPID STAT  NI %CPU %MEM CMD
 1913  1912 Ss     0  0.5  0.0 -bash
 1970  1913 S      0  0.0  0.0 sleep 300
 1971  1913 R      0 97.4  0.0 yes
 1974  1913 R+     0  0.0  0.0 ps -o pid,ppid,stat,ni,%cpu,%mem,cmd
student@okan-lab:~/lab$ pstree -p $$
bash(1913)─┬─pstree(1975)
           ├─sleep(1970)
           └─yes(1971)
```

```
student@okan-lab:~/lab$ kill %1 %2
student@okan-lab:~/lab$ jobs
[1]-  Terminated              sleep 300
[2]+  Terminated              yes > /dev/null
```

What to observe

- -o lists exactly the columns you want: here state (STAT), nice value (NI), %CPU and %MEM.
- yes: R and ≈ 97 % CPU. sleep: S and 0 %.
- pstree -p draws the parent–child tree of Week 3 with real PIDs.
- kill %1 %2 ends both jobs; bash reports "Terminated" at the next prompt.

What the Columns Mean

Column	Meaning	Example
PID / PPID	process ID / parent's process ID (Week 3)	1970 / 1913
UID, USER	owner of the process (Week 11)	student
STAT	state letter + flags (previous slides)	S, R+, Ss
%CPU	CPU time ÷ elapsed time since start, in percent	99.0 (yes)
%MEM	resident memory ÷ total RAM, in percent	0.0
VSZ / RSS	virtual size / resident set (in RAM), in KiB (Weeks 6–7)	3132 / 1884
TTY	terminal the process belongs to; ? = none (daemon)	pts/0
TIME	total CPU time used so far	0:02
CMD, COMMAND	the command line	sleep 300

%CPU above 100 is possible for a multithreaded process: 200 % = two cores fully busy (Week 4 threads).

Everybody's Processes: ps -ef and ps aux

```
student@okan-lab:~/lab$ ps -ef | head -4
UID          PID  PPID  C STIME TTY          TIME CMD
root           1     0  0 16:17 ?        00:00:02 /process_api --firecracker-init …
root           2     0  0 16:17 ?        00:00:00 [kthreadd]
root           3     2  0 16:17 ?        00:00:00 [pool_workqueue_release]
# ... ps -f -u student: only the user student
student@okan-lab:~/lab$ ps -f -u student
UID          PID  PPID  C STIME TTY          TIME CMD
student     1913  1912  0 16:58 pts/0    00:00:00 -bash
student     1970  1913  0 16:58 pts/0    00:00:00 sleep 300
student     1971  1913 99 16:58 pts/0    00:00:01 yes
student     1978  1913  0 16:58 pts/0    00:00:00 ps -f -u student
student@okan-lab:~/lab$ ps aux | head -1
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
student@okan-lab:~/lab$ ps aux | grep '^student'
student     1913  0.6  0.0   8436  7532 pts/0    Ss   16:58   0:00 -bash
student     1970  0.0  0.0   3132  1996 pts/0    S    16:58   0:00 sleep 300
student     1971 99.0  0.0   3132  1884 pts/0    R    16:58   0:02 yes
student     1981  0.0  0.0   8340  4328 pts/0    R+   16:58   0:00 ps aux
student     1982  0.0  0.0   4100  2200 pts/0    S+   16:58   0:00 grep ^student
```

Two styles, same data

- **-ef:** every process, full format (System V style).
- **aux:** every process, with %CPU, %MEM, VSZ, RSS (BSD style, no dash).
- **-u student:** only one user.
- PID 1 is the first process; kernel threads appear [in brackets].
- grep finds itself!

/proc/<pid>/status: Reading the PCB

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2118
student@okan-lab:~/lab$ echo $!
2118
student@okan-lab:~/lab$ grep -E '^(Name|State|Pid|PPid|Uid|Threads|VmRSS)' …
Name:   sleep
State:  S (sleeping)
Pid:    2118
PPid:   2061
Uid:    1013    1013    1013    1013
VmRSS:      2004 kB
Threads:        1
student@okan-lab:~/lab$ grep ctxt /proc/$!/status
voluntary_ctxt_switches:        1
nonvoluntary_ctxt_switches:     2
student@okan-lab:~/lab$ cat /proc/$!/cmdline | tr '\0' ' '; echo
sleep 300
student@okan-lab:~/lab$ ls -l /proc/$!/cwd /proc/$!/exe
lrwxrwxrwx 1 student student 0 Oct 12 16:58 /proc/2118/cwd -> /home/student…
lrwxrwxrwx 1 student student 0 Oct 12 16:58 /proc/2118/exe -> /usr/bin/sleep
```

PCB fields of Week 3

- **State:** S (sleeping) = waiting.
- **Pid, PPid:** identity and parent.
- **Uid:** the owner (Week 11).
- **VmRSS:** memory information.
- **ctxt_switches:** accounting — voluntary = it gave up the CPU, nonvoluntary = it was preempted.
- **cwd, exe:** current directory and program file.

top: A Live View, Refreshed Every Few Seconds

```
student@okan-lab:~/lab$ top -b -n 1 -u student | head -15
top - 16:58:44 up 41 min,  0 user,  load average: 0.59, 0.34, 0.23
Tasks:  71 total,   2 running,  69 sleeping,   0 stopped,   0 zombie
%Cpu(s): 28.6 us, 23.8 sy,  0.0 ni, 42.9 id,  0.0 wa,  0.0 hi,  0.0 si,  4.8 st
MiB Mem :   8029.6 total,   7294.2 free,    598.8 used,    357.6 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   7430.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2260 student   20   0    3132   1876   1760 R 100.0   0.0   0:30.83 yes
 2204 student   20   0    8436   7528   3552 S   0.0   0.1   0:00.01 bash
 2261 student   20   0    3132   2004   1892 S   0.0   0.0   0:00.00 sleep
 2264 student   20   0    9380   5436   3308 R   0.0   0.1   0:00.00 top
 2265 student   20   0    3144   1956   1840 S   0.0   0.0   0:00.00 head
```

Line 1

uptime and load average: 1, 5 and 15-minute averages of the number of processes running or waiting for a CPU.

Line 2–3

Tasks by state (from STAT); %Cpu(s) of the WHOLE machine: us = user mode, sy = kernel mode (Week 2), id = idle.

Process list

PR = 20 + NI (Week 5 priority), S = state, %CPU in the last interval: yes uses 100 % of one core.

WORKED EXAMPLE

Reading the Load Average

Load = average number of processes that are running or ready (R), plus those in disk wait (D). Compare it with the number of CPU cores.

```
student@okan-lab:~/lab$ cat /proc/loadavg
0.59 0.34 0.23 3/98 2262
student@okan-lab:~/lab$ nproc
2
student@okan-lab:~/lab$ uptime
 16:58:45 up 41 min,  0 user,
          load average: 0.59, 0.34, 0.23
```

One yes needs one core: top shows 100 % for it, but only about 50 % in %Cpu(s), the average over 2 cores.

```
cores c = 2  (nproc)

load 0.59 → 0.59 / 2 = 29.5 % of capacity
load 2.00 → 2.00 / 2 = 100 %: every core busy
load 3.00 → 150 %: on average 3 − 2 = 1
            process waits in the ready queue

# 1-min > 15-min: load is rising
# 1-min < 15-min: load is falling
```

Week 4 link: load > cores means processes spend time in the ready queue — their waiting time grows.

CHEAT SHEET

Inspecting Processes at a Glance

Command	What it shows
ps · **ps -f**	processes of this terminal (short / full format)
ps -ef · **ps aux**	all processes of all users
ps -u student	all processes of one user
ps -o pid,ppid,stat,ni,%cpu,cmd	exactly the columns you choose
ps … --sort=-%cpu	sorted, highest CPU first
pgrep -l name · **pgrep -c name**	PIDs (and names) / count of matching processes
pstree -p · **pstree -p $$**	the process tree with PIDs
top · **htop**	live view; q quits
cat /proc/<pid>/status	the PCB as text
echo $$ · **echo $!**	PID of this shell / of the last background job

PART 2

Job Control

One terminal, many jobs: foreground, background, stopped.

Foreground and Background Jobs

Only ONE job owns the keyboard: the foreground job. Ctrl+C and Ctrl+Z are delivered to it.

Foreground: command

- The shell waits (wait() of Week 3) — no prompt until the job ends.
- The job reads your keyboard.
- Ctrl+C sends SIGINT: usually terminates it.
- Ctrl+Z sends SIGTSTP: stops it (state T) and returns the prompt.
- Use it for short commands and anything interactive (nano, less, top).

Background: command &

- The shell prints [job] PID and the prompt at once.
- You keep working while the job runs.
- Ctrl+C does not reach it: use kill %n.
- Its output still appears on your screen — redirect it (> file, Week 12).
- Use it for long, non-interactive work: compiling, copying, downloads.

jobs lists the jobs of THIS shell. fg %n: bring to the foreground. bg %n: continue a stopped job in the background.

Ctrl+Z, jobs and bg

```
student@okan-lab:~/lab$ sleep 100
^Z
[1]+  Stopped                 sleep 100
student@okan-lab:~/lab$ jobs
[1]+  Stopped                 sleep 100
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
 2345 Ss   -bash
 2401 T    sleep 100
 2402 R+   ps -o pid,stat,cmd
student@okan-lab:~/lab$ bg
[1]+ sleep 100 &
student@okan-lab:~/lab$ jobs -l
[1]+  2401 Running                 sleep 100 &
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
 2345 Ss   -bash
 2401 S    sleep 100
 2403 R+   ps -o pid,stat,cmd
```

Step by step

- **sleep 100:** foreground — no prompt.
- **Ctrl+Z (^Z):** stopped; the prompt is back.
- **ps:** STAT T — the stopped state.
- **bg:** continue in the background; the shell adds the &.
- **jobs -l:** now Running, with its PID.
- **ps:** STAT S again — sleep is waiting for its timer, not stuck.

Starting with &, fg, Ctrl+C and kill %n

```
student@okan-lab:~/lab$ sleep 200 &
[2] 2404
student@okan-lab:~/lab$ yes > /dev/null &
[3] 2405
student@okan-lab:~/lab$ jobs
[1]   Running                 sleep 100 &
[2]-  Running                 sleep 200 &
[3]+  Running                 yes > /dev/null &
student@okan-lab:~/lab$ fg %3
yes > /dev/null
^C
student@okan-lab:~/lab$ jobs
[1]-  Running                 sleep 100 &
[2]+  Running                 sleep 200 &
student@okan-lab:~/lab$ kill %1 %2
student@okan-lab:~/lab$ jobs
[1]-  Terminated              sleep 100
[2]+  Terminated              sleep 200
student@okan-lab:~/lab$ jobs
```

Step by step

- **&:** [2] 2404 = job number 2, PID 2404.
- **jobs:** + is the current job, - the previous.
- **fg %3:** yes comes to the foreground; the shell prints its command.
- **Ctrl+C (^C):** SIGINT ends it.
- **kill %1 %2:** % means job number.
- Terminated is reported once, just before the next prompt.

Naming Jobs and Processes

You write	It means	Example
%1, %2 ...	job number n of this shell	fg %2, kill %1
%+ or %%	the current job (marked +); also fg/bg with no argument	fg
%-	the previous job (marked -)	bg %-
%sleep	the job whose command starts with sleep	kill %sleep
1970	a PID — works in any terminal, with any tool	kill 1970
$$	PID of the current shell	echo $$ → 1913
$!	PID of the most recent background job	kill $!

Job numbers (%n) exist only inside the shell that started the jobs. PIDs are global: ps, top and kill in another terminal see them too.

INTERMISSION

10-Minute Break

Next: signals — how kill really works — then priorities and zombies.

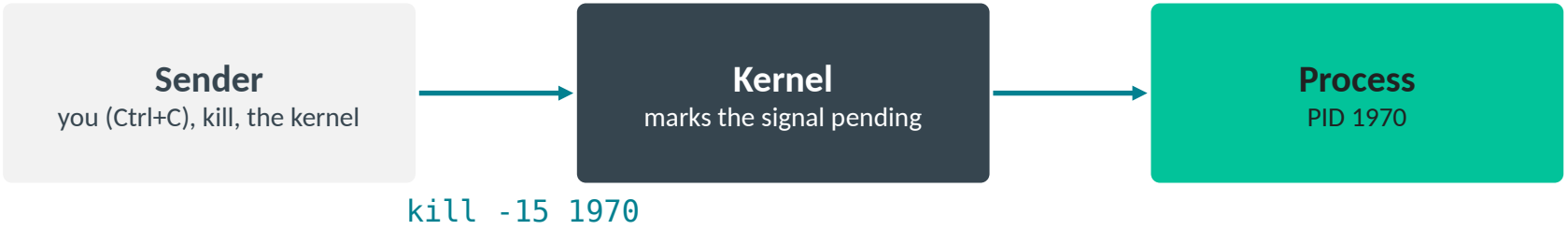
PART 3

Signals

Small numbered messages that the kernel delivers to a process.

A Signal Is a Software Interrupt for a Process

Week 2: a hardware interrupt stops the CPU and runs a handler. A signal does the same for one process.



What the process does with it

- **Default action:** terminate, stop, continue, or ignore — fixed per signal.
- **Catch it:** run its own handler function (e.g. save work, then exit).
- **Ignore it:** the signal has no effect.
- **Exceptions:** SIGKILL (9) and SIGSTOP (19) cannot be caught or ignored.

Rules

- A user may signal only their OWN processes (root: any).
- kill does not always kill: it sends a signal. The default signal is 15, SIGTERM.
- Signals have names and numbers: kill -l.

The Signals You Need

No.	Name	Sent by	Default action	Catchable?
1	SIGHUP	closing the terminal ("hang up")	terminate	yes
2	SIGINT	Ctrl+C	terminate	yes
9	SIGKILL	kill -9 — last resort	terminate, immediately	NO
15	SIGTERM	kill (the default)	terminate	yes
17	SIGCHLD	kernel, when a child ends	ignore	yes
18	SIGCONT	fg, bg, kill -CONT	continue if stopped	yes
19	SIGSTOP	kill -STOP	stop	NO
20	SIGTSTP	Ctrl+Z	stop	yes

Ctrl+\ sends SIGQUIT (3): like SIGINT, but may also write a core dump. SIGSEGV (11) is the kernel's answer to an illegal memory access (Week 7).

kill -l, SIGTERM, SIGKILL, SIGINT — and the Exit Status

```
student@okan-lab:~/lab$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
# ... 64 signals in total
student@okan-lab:~/lab$ sleep 300 &
[1] 2541
student@okan-lab:~/lab$ kill $!
student@okan-lab:~/lab$ wait $!
[1]+  Terminated              sleep 300
student@okan-lab:~/lab$ echo $?
143
student@okan-lab:~/lab$ sleep 300 &
[1] 2542
student@okan-lab:~/lab$ kill -9 $!
student@okan-lab:~/lab$ wait $!
[1]+  Killed                  sleep 300
student@okan-lab:~/lab$ echo $?
137
student@okan-lab:~/lab$ sleep 300 &
[1] 2543
student@okan-lab:~/lab$ kill -INT $!
student@okan-lab:~/lab$ wait $!
[1]+  Interrupt               sleep 300
student@okan-lab:~/lab$ echo $?
130
```

What to observe

- kill -l lists all signals; kill -l 15 prints TERM, kill -l KILL prints 9.
- wait $! waits for the job and reports how it ended: Terminated, Killed, Interrupt.
- $? = exit status of the last command.

```
killed by signal n → $? = 128 + n
SIGTERM: 128 + 15 = 143
SIGKILL: 128 +  9 = 137
SIGINT : 128 +  2 = 130
```

Pause and Resume; Signal Processes by Name

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2544
student@okan-lab:~/lab$ kill -STOP $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
 2544 T    sleep 300

[1]+  Stopped                 sleep 300
student@okan-lab:~/lab$ kill -CONT $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
 2544 S    sleep 300
student@okan-lab:~/lab$ kill $!
```

```
student@okan-lab:~/lab$ sleep 401 &
[2] 2548
[1]   Terminated              sleep 300
student@okan-lab:~/lab$ sleep 402 &
[3] 2549
student@okan-lab:~/lab$ yes > /dev/null &
[4] 2550
student@okan-lab:~/lab$ pgrep -u student -l sleep
2548 sleep
2549 sleep
student@okan-lab:~/lab$ pkill -u student sleep
[2]   Terminated              sleep 401
[3]-  Terminated              sleep 402
student@okan-lab:~/lab$ killall -u student yes
[4]+  Terminated              yes > /dev/null
```

pgrep / pkill / killall pick processes by NAME. Always add -u student: without it they also try other users' processes.

A Program That Catches Signals: catch.c

```c
static volatile sig_atomic_t got = 0;
static void handler(int sig) { got = sig; }  /* tiny! */

sa.sa_handler = handler;
sigaction(SIGINT,  &sa, NULL);   /* Ctrl+C, 2: caught */
sigaction(SIGTERM, &sa, NULL);   /* kill,  15: caught */
if (sigaction(SIGKILL, &sa, NULL) < 0)  /* 9: refused */
    perror("catch: sigaction(SIGKILL)");

for (;;) {
    pause();                     /* sleep until a signal */
    if (got == SIGINT)
        printf("... ignoring it, still running\n");
    if (got == SIGTERM) {
        printf("... cleaning up, bye\n");
        return 0;
    }
}
```

SIGINT is caught (only a message); SIGTERM gives a clean exit (Done); a SIGKILL handler is refused, and kill -9 always works.

```
student@okan-lab:~/lab/src$ ./catch
catch: sigaction(SIGKILL): Invalid argument
catch: PID 2694, waiting for signals
^Ccatch: got SIGINT (2) -- ignoring it, still running
^Ccatch: got SIGINT (2) -- ignoring it, still running
^Z
[1]+  Stopped                 ./catch
student@okan-lab:~/lab/src$ bg
[1]+ ./catch &
student@okan-lab:~/lab/src$ kill -INT %1
student@okan-lab:~/lab/src$ catch: got SIGINT (2) -- ignoring it, still …
kill -TERM %1
student@okan-lab:~/lab/src$ catch: got SIGTERM (15) -- cleaning up, bye
jobs
[1]+  Done                    ./catch
student@okan-lab:~/lab/src$ ./catch &
[1] 2695
student@okan-lab:~/lab/src$ catch: sigaction(SIGKILL): Invalid argument
catch: PID 2695, waiting for signals
kill -9 %1
student@okan-lab:~/lab/src$ jobs
[1]+  Killed                  ./catch
```

GOOD PRACTICE

Stopping a Process Safely: Ask First, Force Last

`Ctrl+C`
if it is in the foreground (SIGINT)

→

`kill PID`
SIGTERM: "please finish"; wait 5–10 s

→

`ps -p PID`
still there? check its STAT

→

`kill -9 PID`
SIGKILL: no cleanup, no saving

Why not always kill -9?

- The program cannot save open documents or flush buffers.
- Temporary and lock files stay behind; a database may need repair.
- Its children are not told; they become orphans.

When kill -9 does NOT help

- **Z (zombie):** already dead — only the parent's wait() removes it.
- **D (disk wait):** the signal is delivered when the I/O finishes.
- **Someone else's process:** Operation not permitted (ask root).

Surviving a Closed Terminal: nohup

```
student@okan-lab:~/lab$ echo "my shell is PID $$"
my shell is PID 2775
student@okan-lab:~/lab$ sleep 500 &
[1] 2831
student@okan-lab:~/lab$ echo "last background job is PID $!"
last background job is PID 2831
student@okan-lab:~/lab$ nohup sleep 600 &
[2] 2832
student@okan-lab:~/lab$ nohup: ignoring input and appending output to 'nohup.ou…
nohup sleep 700 > /dev/null 2>&1 &
[3] 2834
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  nohup.out  photos  projects  scripts  src
student@okan-lab:~/lab$ jobs -l
[1]   2831 Running                 sleep 500 &
[2]-  2832 Running                 nohup sleep 600 &
[3]+  2834 Running                 nohup sleep 700 > /dev/null 2>&1 &
student@okan-lab:~/lab$ kill -HUP %1 %2 %3
student@okan-lab:~/lab$ jobs -l
[1]   2831 Hangup                  sleep 500
[2]-  2832 Running                 nohup sleep 600 &
[3]+  2834 Running                 nohup sleep 700 > /dev/null 2>&1 &
student@okan-lab:~/lab$ kill %2 %3
```

What to observe

- $$ = the shell, $! = the last background job.
- Closing the terminal sends SIGHUP (1) to its jobs; we simulate it with kill -HUP.
- Plain sleep 500: Hangup — gone.
- nohup jobs ignore SIGHUP: still Running.
- nohup's output goes to nohup.out unless you redirect it.

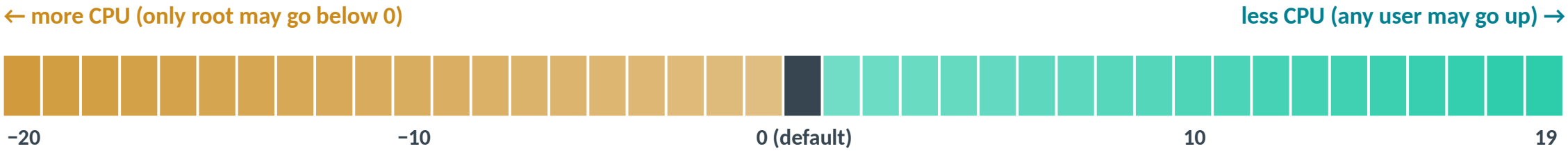
PARTS 4 AND 5

Priorities and Zombies

Week 5 scheduling and Week 3 termination, measured.

LINK TO WEEK 5

nice Values: Being Nice to the Other Processes



Week 5 recap

- Smaller number = higher priority, as in Week 5.
- Linux shares the CPU by weight: nice 0 → 1024, nice 10 → 110; each step ≈ × 1.25.
- top shows PR = 20 + NI.

```
# two CPU-bound jobs on ONE core
share(nice 0)  = 1024 / (1024 + 110)
               = 90.3 %
share(nice 10) =  110 / (1024 + 110)
               =  9.7 %
ratio          = 1024 / 110 = 9.31
check: 1.25^10 = 9.31
```

nice and renice in Practice

```
student@okan-lab:~/lab$ nice
0
student@okan-lab:~/lab$ nice -n 10 yes > /dev/null &
[1] 2971
student@okan-lab:~/lab$ ps -o pid,ni,stat,cmd
  PID  NI STAT CMD
 2914   0 Ss   -bash
 2971  10 RN   yes
 2972   0 R+   ps -o pid,ni,stat,cmd
student@okan-lab:~/lab$ renice -n 15 -p $!
2971 (process ID) old priority 10, new priority 15
student@okan-lab:~/lab$ renice -n 5 -p $!
renice: failed to set priority for 2971 (process ID): Permission denied
student@okan-lab:~/lab$ nice -n -5 sleep 1
nice: cannot set niceness: Permission denied
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ taskset -c 1 yes > /dev/null &
[2] 2976
[1]   Terminated              nice -n 10 yes > /dev/null
student@okan-lab:~/lab$ taskset -c 1 nice -n 10 yes > /dev/null &
[3] 2977
student@okan-lab:~/lab$ ps -o pid,ni,stat,%cpu,cmd
  PID  NI STAT %CPU CMD
 2914   0 Ss    0.0 -bash
 2976   0 R    89.3 yes
 2977  10 RN    9.6 yes
 2978   0 R+    0.0 ps -o pid,ni,stat,%cpu,cmd
student@okan-lab:~/lab$ kill %2 %3
```

What to observe

- nice alone prints the current value: 0.
- nice -n 10: NI 10, STAT RN (N = niced).
- renice up (10 → 15): allowed. Down (15 → 5): Permission denied.
- Two yes on one core (taskset -c 1): 89.3 % vs 9.6 %, ratio 9.30 — predicted 9.31.

zombie.c: A Child That Nobody Waits For

```
pid_t pid = fork();                 /* one process becomes two */

if (pid == 0) {                     /* ---- child ---- */
    printf("child  %d: exiting now with status 7\n", getpid());
    exit(7);                        /* terminated -> zombie until reaped */
}
/* ---- parent: does NOT wait() for secs seconds ---- */
printf("parent %d: child is %d, sleeping %d s without wait()\n",
       getpid(), pid, secs);
sleep(secs);                        /* meanwhile the child is a zombie */

waitpid(pid, &status, 0);           /* collect the exit status: reap */
printf("parent %d: reaped child %d, exit status %d\n",
       getpid(), pid, WEXITSTATUS(status));
```

The plan

- fork(): parent and child (Week 3).
- The child exits at once with status 7.
- The parent sleeps 25 s WITHOUT wait(): the child is a zombie meanwhile.
- Then waitpid() collects status 7: the zombie is reaped.

Seeing State Z — and Failing to Kill It

```
student@okan-lab:~/lab/src$ gcc -Wall -O2 -o zombie zombie.c
student@okan-lab:~/lab/src$ ./zombie 25 &
[1] 3119
student@okan-lab:~/lab/src$ parent 3119: child is 3120, sleeping 25 s without w…
child  3120: exiting now with status 7
ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3119  3057 S    ./zombie 25
 3120  3119 Z    [zombie] <defunct>
 3121  3057 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab/src$ grep State /proc/$(pgrep -u student -n zombie)/stat…
State:  Z (zombie)
student@okan-lab:~/lab/src$ kill -9 $(pgrep -u student -n zombie)
student@okan-lab:~/lab/src$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3119  3057 S    ./zombie 25
 3120  3119 Z    [zombie] <defunct>
 3125  3057 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab/src$ wait
parent 3119: reaped child 3120, exit status 7
[1]+  Done                    ./zombie 25
student@okan-lab:~/lab/src$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3128  3057 R+   ps -o pid,ppid,stat,cmd
```

What to observe

- PID 3120: STAT Z, [zombie] <defunct>, PPID 3119.
- /proc says: State Z (zombie).
- kill -9 does nothing: the process is already dead.
- After the parent's waitpid(): exit status 7, and the zombie is gone.

CHEAT SHEET

Controlling Jobs and Processes at a Glance

Command / key	Effect
command &	start in the background; prints [job] PID
Ctrl+C · Ctrl+Z	SIGINT to the foreground job · SIGTSTP: stop it
jobs · jobs -l	list this shell's jobs (with PIDs)
fg %n · bg %n	to the foreground · continue in the background
kill PID · kill %n	SIGTERM (15) to a process / a job
kill -9 · -STOP · -CONT	SIGKILL (last resort) · pause · resume
pkill -u student name · killall	signal processes by name
nohup command > log 2>&1 &	keep running after the terminal closes
nice -n 10 cmd · renice -n 15 -p PID	start with / change the nice value (users: up only)

IN-CLASS LAB · 30–40 MIN

Lab Exercises (1/2): Inspect and Control

A · Inspect (1–5)

- echo $$ and find it in ps
- ps -f: who is the parent of ps?
- sleep 1000 &: State and PPid from /proc/$!/status
- pstree -p $$ with two sleeps
- R versus S: yes and sleep side by side

B · Job control (6–9)

- sleep 500, Ctrl+Z, jobs, STAT T
- bg; check with jobs -l and ps
- three jobs; fg %2, then Ctrl+C
- kill %1 versus kill 1: explain

Work in ~/lab (run bash mcmp117_lab_setup.sh first). Write down every command and its output: the Lab Sheet in the notes has all 17 exercises and the point values.

IN-CLASS LAB · 30–40 MIN

Lab Exercises (2/2): Signals, Priority, Zombies

C · Signals (10–13)

- $? after SIGTERM, SIGKILL, SIGINT: explain 128 + n
- STOP and CONT: watch STAT
- pgrep -c, pkill -u student
- nohup versus plain job: kill -HUP both

D · Priority, zombies, challenge (14–17)

- nice -n 15, renice to 19, then to 0
- ./zombie 30 &: find Z, try kill -9
- top 3 processes by %CPU (--sort)
- count your processes per state with a Week 12 pipeline

Clean up when you finish: jobs must be empty and pgrep -u student -l sleep must print nothing.

Error Messages and What They Mean

You see	Why	Fix
-bash: kill: (1) - Operation not permitted	kill 1 means PID 1 (system!)	kill %1 for job 1
-bash: kill: %6: no such job	job numbers are per shell	check jobs first
renice: ... Permission denied	users may not lower nice	only raise it (or root)
nice: cannot set niceness: Permission denied	negative nice as a user	runs at nice 0 anyway
the prompt does not come back	a foreground job is running	Ctrl+C, or Ctrl+Z then bg
[1]+ Stopped ... (tty input)	background job wants the keyboard	fg %1
Z <defunct> survives kill -9	zombies are already dead	end or fix the parent

Read the message before retyping: bash, kill, renice and nice each tell you exactly which rule you broke.

QUICK QUIZ

Check Your Understanding

1. ps shows STAT T for your process. Which Week 3 state is closest, and which two commands bring it back?

2. A job ended and echo $? prints 137. What happened to it? What would 143 mean?

3. Two CPU-bound processes on one core have nice 0 and nice 10. Roughly what CPU share does each get?

4. Why can kill -9 not remove a zombie? What does?

WRAP-UP

Week 13 — Key Takeaways

- **Processes:** every command is a process with a PID and PPID, created by the shell with fork + exec (Week 3).
- **Looking:** ps (-f, -ef, aux, -o), pstree, top/htop, /proc/<pid>/status (the PCB).
- **STAT:** R = ready/running, S/D = waiting, T = stopped, Z = terminated but not reaped.
- **Job control:** & starts in the background; Ctrl+Z stops; jobs, fg, bg; %n is a job, a plain number a PID.
- **Signals:** kill sends SIGTERM (15) by default; Ctrl+C = SIGINT (2); kill -9 = SIGKILL, last resort; $? = 128 + n.
- **Priority:** nice −20…19, users may only raise it; nice 10 vs 0 ≈ 1 : 9.3 of the CPU (Week 5).
- **Zombies:** only the parent's wait() removes them.

NEXT WEEK

Introduction to Shell Scripting

Week 14 will cover:

- Why script: automate what you typed today
- Shebang, chmod +x, variables, quoting, $( ) and $(( ))
- $0 $1 $# $@, read, exit status $? and exit
- test / [ ], if, for, while, functions, bash -x
- Two real scripts: backup.sh and report.sh in ~/lab/scripts

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz